50.051 Programming Language Concepts

Final Report

Group 8

Name	Student ID
Foong Jingxuan Janelle	1008146
Hoon Wei Yang	1007784
Skipper Smahon	1008162
Ong Wei Yang	1008126
Finn Yap Jun Hun	1008122

1. Introduction **3**
2. Working with Constraints **3**
2.1. File Reading and Writing 3
2.2. Custom Parser 3
2.3. Finite State Machines (FSMs) 3
3. System Overview **4**
3.1 System Inputs and Outputs 4
3.2 System Architecture 4
3.3 State Machines 4
4. Features **6**
4.1 File Reading and Writing 6
4.2 Parser 6
4.3 Tokenizer 6
4.4 Evaluator 7
4.5 Game UI 7
4.6 Game Logic 8
4.7 Leaderboard 9
4.8 Replay 10
4.9 Challenge Mode 10
5. Development Process **10**
5.1 Clean C Programming 10
5.2 Work DIstribution 11
5.3 Challenges Faced 11
5.4 Testing and Cross-Platform Verification 12
6. Running the Project **12**
7. Reflections **12**
7.1. What We Learned 12
7.2. What We Would Do Differently 13
8. Repository **13**

1. Introduction

NERDLE is a number-based twist on the popular word game Wordle. Instead of guessing hidden words, players must deduce a concealed mathematical equation within six attempts.

The core challenge lies in the real-time validation of player input. Unlike a simple word-matching game, every guess in NERDLE must form a mathematically valid identity. This required building a robust Custom Syntax Parser capable of handling operator precedence, prevent division by zero, and ensure that the left-hand side strictly equals to the right-hand side of the equation before the guess is even processed by the game engine.

On top of validation, the game provides Wordle-style feedback in the form of colored cells: green for a correct character in the correct position, yellow for a correct character in the wrong position, and red for an absent character. This gives players the information they need to iteratively narrow down the hidden equation.

This project was chosen because the team wanted something that was simple to understand but could test C fundamentals. The game runs entirely in the terminal and is written in pure C with no external libraries.

2. Working with Constraints

2.1. File Reading and Writing

NERDLE reads mathematical equations from a plaintext (.txt) file which we have populated at the start of each game session, randomly selecting one as the hidden answer. Players can also contribute new equations through the game menu (Selection 3), which are validated and appended to the same file, ensuring the equation bank grows over time.

2.2. Custom Parser

NERDLE's custom-built syntax parser validates the player's guess before it is processed by the game engine. It uses a FSM to walk through each character of the input, enforcing structural rules such as correct operator placement, mandatory equality, and digit sequencing. It also evaluates both sides of the equation arithmetically, accounting for operator precedence and rejecting inputs when both sides are unequal.

2.3. Finite State Machines (FSMs)

Two FSMs drive the core logic of NERDLE. The first is embedded in the parser, transitioning through states such as *STATE_START*, *STATE_NUM_LEFT*, *STATE_OPERATOR*, *STATE_EQUALS*, *STATE_NUM_RIGHT*, and *STATE_INVALID* to validate equation structure character by character. The second FSM governs the game flow itself, managing transitions between states consisting of waiting for input, validating a guess, and determining win or loss conditions.

3. System Overview

3.1 System Inputs and Outputs

The program loads a bank of valid equations from a text file, randomly selects one as the hidden answer, and runs a guessing loop where the player has six attempts to identify the correct answer. After each guess, the player receives colour-coded feedback indicating which characters are correct, misplaced, or absent. After each game, the result is saved to the leaderboard and a replay file is generated. The player interacts entirely through the terminal.

Inputs:

- *equations.txt* - a text file containing valid equations (e.g. 3+5*2=13)
- Player keyboard input during gameplay

Outputs:

- Colour-coded terminal feedback after each guess
- *scores.txt* - a text file storing game results and statistics
- *leaderboard.txt* - a text file containing records of players who completed a game
- **.nrdl* replay files stored in the replays/ folder

3.2 System Architecture

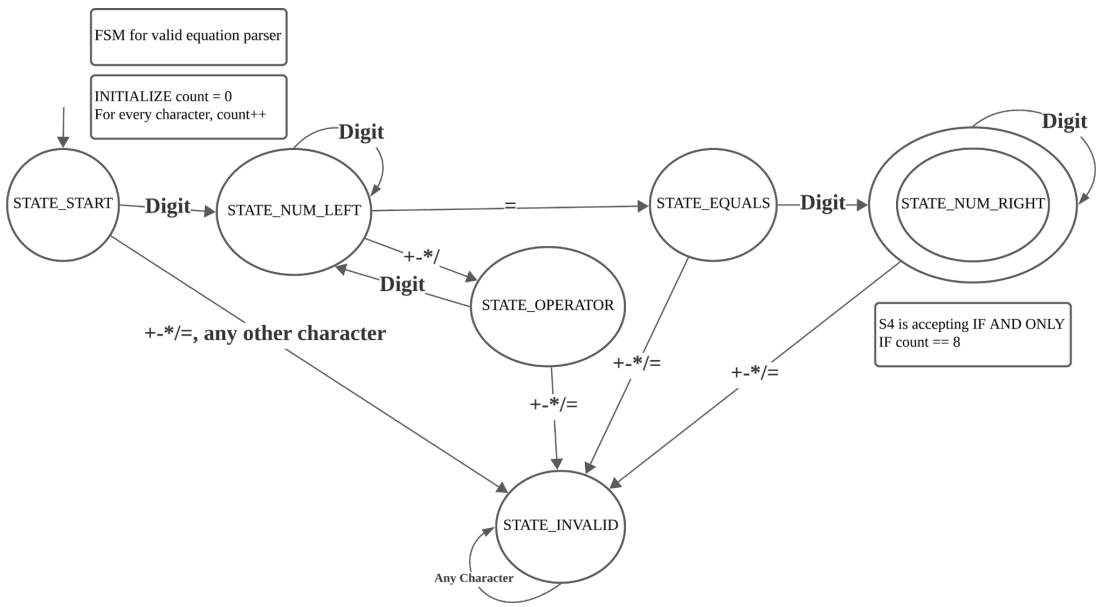
The program is structured around five main components: file I/O, equation parsing and evaluation, tokenization, the game state machine, and the terminal user interface. Each component is implemented in its own .c and .h file pair, keeping the codebase modular and maintainable. The main.c file serves only as the entry point and top-level coordinator, delegating all logic to the appropriate module.

3.3 State Machines

Two FSMs were implemented, equation validation and game flow.

The equation utilizes this FSM with a count state:

State	Description
Start	Starting state
Digit (Left)	Reading a digit on the left-hand side
Operator	Reading an operator (+, -, /, *)
Equals	Reading the = sign
Digit	Reading a digit on the right-hand side
Invalid	Sink state – once entered, cannot be left

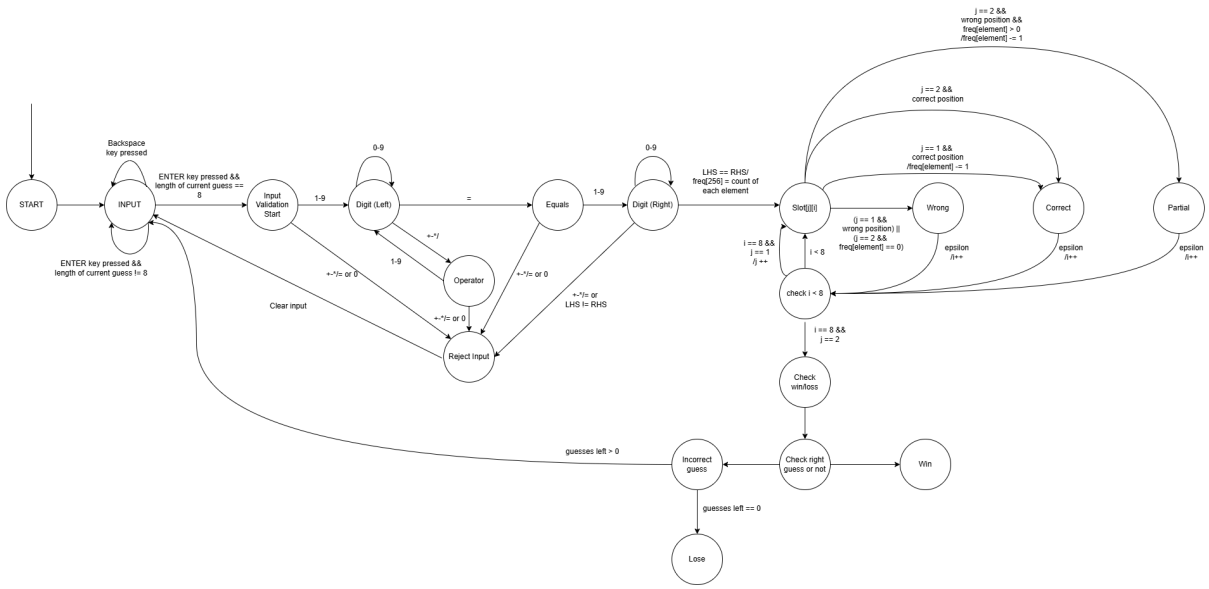


[For enlarged display]

Having an external state is not a traditional thing in FSMs, however it was implemented to reduce the complexity of the state machine by a big factor. If this was a traditional state machine without the count state which incremented every character, we would require a grid of states to match for the 8 characters in the equation.

The game loop is driven by a larger FSM, incorporating the equation validation FSM on top of the following additional states:

State	**Description**
Slot[j][i]	Unevaluated character in pass *j*, slot *i*
Wrong	Pass *j*==1, character in wrong position; Pass *j*==2, no remaining character
Partial	Pass *j*==2, character in wrong position
Correct	Character in correct position
Check *i*<8	Verify position of current “pointer” and check current pass
Check win/lose	Intermediate checking state
Check guess	Verify if input equation matches target equation
Win	Input equation matches target equation
Incorrect guess	Input equation does not match target equation
Lose	No more guessing attempts



[For enlarged display]

4. Features

4.1 File Reading and Writing

Both binary and text files are utilized across the program. The equation bank is stored as a plain text file (*equations.txt*), read at startup using standard C file I/O. The leaderboard is a text file (*leaderboard.txt*) containing records of players who have completed the game. Each record contains player name, date, time of recording, and time taken to complete an equation. Records are sorted according from fastest to slowest timings. Player scores are read from a text file and session history are written to and read from a binary file (**.nrdl*), which stores the game state of a game and replays it.

4.2 Parser

The parser validates player inputs before they are accepted as guesses, checking that:

- The equation follows the format [expression]=[result]
- Only valid characters are used (digits 0–9 and operators +, -, *, /, =)
- The input is exactly [N] characters long

Any input that fails these checks is rejected before it reaches the game engine, keeping the validation logic cleanly separated from gameplay.

4.3 Tokenizer

The tokenizer tokenizes the parsed equation (raw string) into tokens (digits and operators). This is done in a single scan by checking whether the character is a digit or an operator. If it is a digit, the tokenizer will continue checking for digits and concatenating into a token, with attribute is_number = 1. If it is a space, the tokenizer will ignore and move on to the next character. If it is a token, the tokenizer will create a new token with attribute is_number = 0.

4.4 Evaluator

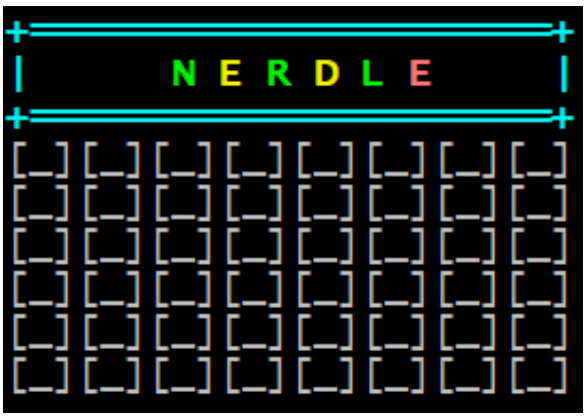
The evaluator evaluates the tokens created from the string from the tokenizer to check whether an equation is valid. Validity is defined by having the left hand side be equivalent to the right hand side after all the operators have been applied appropriately (LHS = RHS) and no division of 0. This is accomplished by first taking the tokens which were infix by nature and turning them into postfix expressions through the Shunting-Yard algorithm, which accounts for precedence (*, / having higher precedence than +, -). This was the chosen direction as the team felt that a postfix evaluation was the most straightforward approach to this function.

Subsequently, postfix expression evaluation is conducted by utilizing a stack to store digit tokens upon receiving them (is_number == 1), and popping two digit tokens upon receiving an operator (is_number == 0). At this point, division of 0 is verified. If the operator was a '/' and the number on the right hand side of the operator is a 0, the evaluator returns -1, which is propagated up as an error.

4.5 Game UI

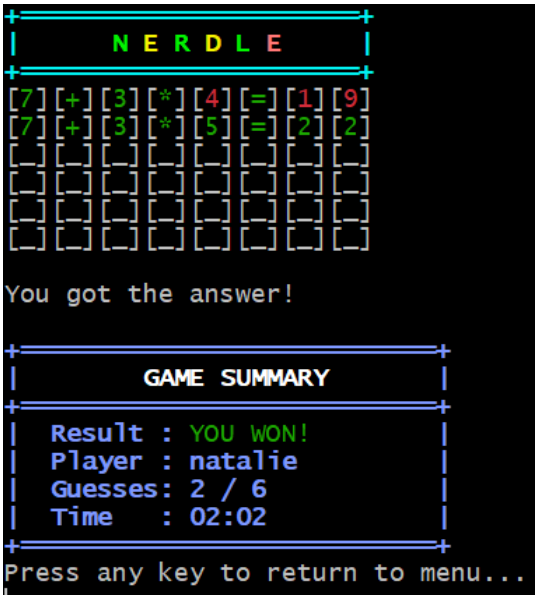
For the menu and header system, the game UI provides a standardized aesthetic for the application using structured borders and a distinct color palette (cyan and blue) to differentiate between the main navigation and specific sub-sections.

The game UI serves as the view layer of our NERDLE application, responsible for managing the Terminal User Interface (TUI) and visual feedback. The guess board is the core visual element is a dynamic grid that renders the player's attempts. It provides real-time color-coded feedback for each character: green for the correct character in the correct position, yellow for the correct character but in the wrong position, and red for character not present in the target equation.



4.6 Game Logic

Reusing the equation validation logic from the parser as mentioned above, the game logic ties together user input, input validation, guess evaluation and game result conclusion in a state machine to manage a complete game session. It maintains the list of guesses made, tracks remaining attempts, and computes per-character feedback by comparing each guess against the hidden answer. The feedback logic categorises each character as correct (right character, right position), partial (right character, wrong position), or absent, mirroring the classic Wordle feedback model.



Notably, this logic must be achieved via iterating through the input string twice. Upon receiving the target equation, the game stores the count of each character in an array *freq[256]*, with each index representing the count of the corresponding ASCII character. In the first pass (*j*==1), characters are only assigned Correct or Wrong based on their position, Correct characters invoke a decrement to the corresponding count in *freq*. On the second pass (*j*==2), Wrong characters are re-evaluated to Partial if their corresponding count in *freq* is more than 0 (*freq[x]*>0).

If the player did not manage to solve the hidden equation within 6 attempts, the output will be as such:

```
+=========================+
|        N E R D L E      |
+=========================+
[1][2][+][2][1][=][3][3]
[1][2][+][2][2][=][3][4]
[1][2][+][1][2][=][2][4]
[4][5][+][3][4][=][7][9]
[1][0][+][1][0][=][2][0]
[8][8][+][1][1][=][9][9]

Guesses left: 0

No guesses left. The answer was: 72-36=36

+=============================+
|          GAME SUMMARY       |
+=============================+
|  Result : Try Again         |
|  Player : jane              |
|  Guesses: 6 / 6             |
|  Time   : 02:31             |
+=============================+
Press any key to return to menu...
```

4.7 Leaderboard

The leaderboard maintains a record of player performances through the use of a *.txt* file. Each record consists of a struct *LeaderboardEntry* which takes in the player's name, total time taken for their completed playthrough in minutes and seconds, and the date and time at which they completed their equation.

The game starts a timer when a game is started and utilises the leaderboard only when a user wins their game. The game will check the FSM state of the game with the condition is_game_won(game) == 1. At this point, an end timer variable is defined and through the time difference of the start and stop variables, the total time of a user's playthrough is calculated. With the minutes and seconds obtained, the game will call *writeLeaderboard* while taking the player's name, minutes and seconds as inputs. The date and time of the playthrough is automatically calculated in the function itself.

If the total leaderboard entries is below 100 then the leaderboard file will move each entry that is slower up by one index and then places the player's entry into a position where the next index is slower but the previous index before it is a faster entry. This ensures that the leaderboard file is sorted according to completion times. There is also some underlying logic in *writeLeaderboard* which first compares the total time taken in the player's run against the last leaderboard entry if there are 100 leaderboard entries. If the user is faster than the last entry, then the last place on the leaderboard is removed and the user is slotted into their corresponding position according to the logic as described earlier.

LEADERBOARD

Rank	Player	Time	Date	Logged
1	janelle	00:05	19/04/2026	20:11:56
2	Skipper	00:12	13/04/2026	02:23:44
3	finn	00:17	19/04/2026	20:32:40
4	wy	00:53	19/04/2026	20:32:10

4.8 Replay

The replay feature records and plays back completed game sessions using a custom binary file format with the .nrdl extension, stored in a replays folder.

Saving: After each game, *save_replay* writes the session to a binary file with the naming convention: replay_<name>_<timestamp>.nrdl. The file begins with a 4-byte magic number (NRDL) for format validation, followed by a 4-byte integer storing the number of guesses made. Each guess turn is then written sequentially, first, the guessed equation as a string of size (EQUATION_LEN+1) bytes, then, the feedback array of size EQUATION_LEN*sizeof (SlotState) bytes, indicating whether each character was correct, partially correct, or wrong.

Playing back: *play_replay* scans through the replays folder and presents the user with a numbered list of available replay files. Upon selection, the file is opened and validated by checking whether it starts off with "NRDL". The turn count is read, and each turn is reconstructed by reading the guessed equation and feedback array back in the same order they were written, then displayed using the UI functions we created for the game.

The binary format was chosen for compactness and because the SlotState enum values map directly to byte values with no conversion needed.

4.9 Challenge Mode

The main game picks a random equation from *equations.txt*. Challenge mode simply allows users to input a target equation using the Add New Equation functionality, but instead of adding the equation to the text file, it is used as the target equation for a new game. This allows players to challenge friends and family to guess their specifically chosen equations.

5. Development Process

5.1 Clean C Programming

The entire project is written in C and compiles with: `gcc -Wall -Werror -ansi -pedantic`. The code is organized across separate *.c* and *.h* file pairs for each component, keeping each module focused on a single responsibility,making the codebase easy to navigate and extend.

5.2 Work Distribution

Member	Responsibilities
Finn	Tokenizer for evaluator, replay logic
Ong Wei Yang	Game logic, game UI
Skipper	File I/O, leaderboard
Janelle	Custom Parser, game UI
Hoon Wei Yang	Game logic, challenge mode

5.3 Challenges Faced

Equation Validation: For example, a guess like 2+3*2=8 is mathematically correct, but a simple left-to-right evaluation would compute 10 and wrongly reject it. This meant we could not simply scan the expression linearly and had to ensure the postfix conversion was correctly applied for every guess, not just edge cases. We had to take care of precedence between operators, which we chose to tackle with a postfix evaluation algorithm. However this then required us to change the typically infix input from players into a postfix input from players which needed a shunting yard algorithm.

Character Checking Logic: Initially, the logic for verifying a character's correctness was done with a simple FSM, when seeing a character in the correct position, it is labelled Correct; character is in the equation but in the wrong position is labelled Partial; and character not in the equation is labelled Wrong. This seemed fine at first, however during testing, we realised that repeated characters are not accounted for. For example, with a target equation of 12+34=46, a guess of 22+22=44 would yield three yellow (Partial) 2s and one green (Correct) 2. This is not ideal and is incorrect feedback. Similar to Wordle, correct feedback would yield one green (Correct) 2 in position 2, and red (Wrong) 2s for all other 2s. To overcome this issue, the FSM had to be made more complex, incorporating 2 passes through the input string as described in section 4.6.

Cross-platform compatibility: We realized fairly late in development that conio.h, which we had been using for raw keyboard input via _getch(), is a Windows-only header that does not exist on Linux/Mac. This meant that the program compiled and ran correctly on Windows but failed on other systems. To fix this, we created our own input.h and input.c module that wraps the platform difference using preprocessor directives. On Windows, it calls _getch() from conio.h as before. On Linux and Mac, it achieves the same raw keypress behaviour by directly manipulating the terminal settings using termios.h.

Binary file structure: The issue with struct padding that might have come up was avoided as a whole as we wrote each field with explicit sizeof.

Variable management: As we combined one another's parts, there were times when variables we used in our main.c conflicted. It was important to ensure that either separate variables are used or we did not have any conflicts when using the same variable for our various game functions. This is largely due to the fact that in other coding languages, it is easier to define a variable whenever you want.

Initial monolithic code: Initially in implementing the main game logic, all functionality (e.g. game logic itself, UI printing) were written in a single *.c* file, making the code hard to follow and unconducive for future modifications. The single file was modularised later into multiple *.c* files for the separation of concerns.

5.4 Testing and Cross-Platform Verification

The project was tested on Windows 11 and macOS. To make sure it compiles cleanly on everyone's machine, the team used a shared Makefile that all team members ran locally.

6. Running the Project

Refer to the `README.md` in the repository for full instructions. In short:

```
make
./nerdle
```

Ensure that equations.txt is in the same directory as the executable before running. Replay files will be saved automatically to a replays/ folder created on first use.

To see the hidden equation to be solved, uncomment this in game_ui.c (line 156):

```
printf("Target equation (for debugging): %s\n", game->answer);
```

7. Reflections

7.1. What We Learned

Building NERDLE in pure C taught us that problems which seem simple at the application level, like validating a math equation, quickly become nuanced at the implementation level.

File read and write functions are generally much harder to code out in C as compared to other languages like Java, Python, or JavaScript. It makes working with other programming languages more interesting as memory management, pointers, and file parsing are handled in the background that developers don't really need to care about, but if you are aware of those concepts then it is interesting to think how your respective coding language is handling that. It makes you wonder how much abstraction goes on in other programming languages.

The very strict syntax of C especially with the compilation flags was not very comfortable to live with initially. Some issues are not apparent until trying to compile, and simple things like

missing a new liner at the end of a file incur additional overhead when tracing back to the file just to add a new liner before compiling again. Duplicate definitions are also not clear especially to beginners. Reusing the same include statements does not raise issues until after trying to compile, at which point we think the code is ready but it actually is not.

State machines turned out to be genuinely useful for structuring both the parser and the game loop in a way that was easy to reason about and debug. Just utilising mainly built-in C functionalities gives us an understanding on why people tend to not like C, and prefer high-level programming languages. The difficulty of managing memory when it comes to strings is trivialized in other programming languages such as Java, Python etc. However the flexibility of C is what gives the incredible speed and customisation it has.

Initially we thought including a binary file would be a headache, however our proper planning of "classes", or structures to be exact allowed the storing of binary data to be a breeze. We reused our functions we created for the game to create a replay system, showing proper programming with clean separation of concerns leads to better developer experience.

7.2. What We Would Do Differently

We would modularise the code from the very beginning rather than refactoring it later. Starting with a single file made development progress quickly, but the eventual refactor into separate modules cost us time and broke certain features.

We would also write test cases early on, manually testing edge cases like division by zero, single-digit equations, and equations with multiple operators became tedious and error-prone as the codebase grew.

8. Repository

GitHub: https://github.com/how2fps/nerdle-plc